

Temple Management System

Complete Setup Guide

This document covers the full local development setup on **Ubuntu 24.04 LTS** and the production deployment on **AWS EC2 (Ubuntu 26.04)** with the **Nginx** web server.

Tech stack:

- Backend — Django 4.2 + Django REST Framework + uvicorn (ASGI)
- Frontend — React 18 + Vite (built as a static bundle)
- Database — MySQL 8 / MariaDB 10
- Web server — Nginx (reverse proxy)
- Process manager — systemd (or supervisor)

Section	Topic
A	Local Development Setup — Ubuntu 24.04
B	Production Deployment on AWS EC2 — Ubuntu 26.04 + Nginx
C	Environment Files — frontend/.env and backend/.env
D	Post-Install Checklist and Common Issues

A. Local Development Setup — Ubuntu 24.04

This section walks you through setting up the entire stack on a fresh Ubuntu 24.04 laptop or VM for local development and testing. Estimated time: 15 minutes on a decent internet connection.

A.1 Install system dependencies

Open a terminal and run the following in one shot. It installs Python, Node.js (via NodeSource), Yarn, MariaDB and the development headers required for mysqlclient.

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y \
    build-essential pkg-config git curl unzip \
    python3 python3-venv python3-pip python3-dev \
    default-mysql-server default-libmysqlclient-dev \
    nginx

# Node.js 20 LTS via NodeSource
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# Yarn (via npm — most reliable across Ubuntu versions)
sudo npm install -g yarn
```

A.2 Start MariaDB and create the database

```
sudo systemctl enable --now mariadb
sudo mysql_secure_installation # follow prompts, set a root password

# Create the database, user, and grant permissions
sudo mysql <<'SQL'
CREATE DATABASE temple CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
CREATE USER 'appadmin'@'localhost' IDENTIFIED BY 'CHANGE_ME_STRONG';
GRANT ALL PRIVILEGES ON temple.* TO 'appadmin'@'localhost';
FLUSH PRIVILEGES;
SQL
```

Note: If you already have a SQL dump (temple_db.sql), import it now:

```
mysql -u appadmin -p temple < temple_db.sql
```

If the dump was created on MySQL 8, replace utf8mb4_0900_ai_ci with utf8mb4_general_ci before importing (MariaDB compatibility):

```
sed -i 's/utf8mb4_0900_ai_ci/utf8mb4_general_ci/g' temple_db.sql
```

A.3 Clone and configure the project

```
cd ~ && git clone <YOUR_REPO_URL> temple
cd temple
```

```
# Create the two env files (contents in Section C)
nano backend/.env
nano frontend/.env
```

A.4 Backend — Python virtual environment

```
cd ~/temple/backend
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip wheel
pip install -r requirements.txt
```

```
# Migrate (creates tables – safe to run even after a SQL import,  
# it will only apply any missing migrations)  
python manage.py migrate  
python manage.py collectstatic --noinput  
  
# Create your admin superuser  
python manage.py createsuperuser
```

A.5 Frontend — install and run

```
cd ~/temple/frontend  
yarn install  
  
# Dev server with hot reload  
yarn dev          # http://localhost:3000  
  
# OR build a static bundle for production-like preview  
yarn build && yarn add -D http-server && \  
  npx http-server dist -p 3000 -a 0.0.0.0 \  
  -P http://localhost:3000?
```

A.6 Run the backend

```
cd ~/temple/backend && source venv/bin/activate  
uvicorn temple_proj.asgi:application --host 0.0.0.0 --port 8000 --reload
```

Open **http://localhost:3000** in your browser. Sign in with the superuser account you created in step A.4. If you see the sidebar and the Home page, you are done.

B. Production Deployment — AWS EC2, Ubuntu 26.04 + Nginx

B.1 Launch the EC2 instance

In the AWS console:

1. **AMI:** Ubuntu Server 26.04 LTS (arm64 or x86_64).
2. **Instance type:** t3.small minimum (2 GB RAM). For > 500 members use t3.medium.
3. **Storage:** 20 GB gp3 (raise later if uploads grow).
4. **Security group:** allow inbound 22/tcp from your IP, 80/tcp and 443/tcp from anywhere (0.0.0.0/0).
5. **Key pair:** create or reuse one — save the .pem file.
6. **Elastic IP:** allocate and associate so the public IP does not change on stop/start.

B.2 First-time server setup

```
ssh -i temple.pem ubuntu@<ELASTIC_IP>

sudo apt update && sudo apt upgrade -y
sudo timedatectl set-timezone Asia/Kolkata
sudo apt install -y \
    build-essential pkg-config git curl unzip ufw \
    python3 python3-venv python3-pip python3-dev \
    mysql-server default-libmysqlclient-dev \
    nginx certbot python3-certbot-nginx

# Node.js 20 LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs && sudo npm i -g yarn

# Firewall
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw --force enable
```

B.3 MySQL setup

```
sudo systemctl enable --now mysql
sudo mysql_secure_installation

sudo mysql <<'SQL'
CREATE DATABASE temple CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
CREATE USER 'appadmin'@'localhost' IDENTIFIED BY 'REPLACE_WITH_STRONG_PW';
GRANT ALL PRIVILEGES ON temple.* TO 'appadmin'@'localhost';
FLUSH PRIVILEGES;
SQL
```

B.4 Deploy the code

```
sudo mkdir -p /opt/temple && sudo chown ubuntu:ubuntu /opt/temple
cd /opt/temple
git clone <YOUR_REPO_URL> app
cd app

# ---- Backend ----
cd backend
python3 -m venv venv && source venv/bin/activate
pip install --upgrade pip wheel
```

```

pip install -r requirements.txt gunicorn uvicorn
nano .env      # paste the production backend/.env from Section C
python manage.py migrate
python manage.py collectstatic --noinput
python manage.py createsuperuser
deactivate

# ---- Frontend ----
cd /opt/temple/app/frontend
nano .env      # paste the production frontend/.env from Section C
yarn install
yarn build     # generates dist/

```

B.5 systemd service for the backend

Create /etc/systemd/system/temple-backend.service:

```

[Unit]
Description=Temple Management Django (uvicorn)
After=network.target mysql.service
Requires=mysql.service

[Service]
User=ubuntu
Group=www-data
WorkingDirectory=/opt/temple/app/backend
EnvironmentFile=/opt/temple/app/backend/.env
ExecStart=/opt/temple/app/backend/venv/bin/uvicorn \
    temple_proj.asgi:application \
    --host 127.0.0.1 --port 8000 \
    --workers 3 --proxy-headers \
    --forwarded-allow-ips='*'
Restart=always
RestartSec=5
StandardOutput=append:/var/log/temple/backend.out.log
StandardError=append:/var/log/temple/backend.err.log

[Install]
WantedBy=multi-user.target

sudo mkdir -p /var/log/temple && sudo chown ubuntu:www-data /var/log/temple
sudo systemctl daemon-reload
sudo systemctl enable --now temple-backend
sudo systemctl status temple-backend

```

B.6 Nginx reverse proxy + static frontend

Create /etc/nginx/sites-available/temple:

```

server {
    listen 80;
    server_name temple.example.com www.temple.example.com;

    client_max_body_size 25M;    # allow member-photo uploads

    # ---- Backend API + admin + media ----
    location /api/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;
        proxy_set_header Host                $host;
        proxy_set_header X-Real-IP           $remote_addr;
        proxy_set_header X-Forwarded-For    $proxy_add_x_forwarded_for;
    }
}

```

```

        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Upgrade          $http_upgrade;
        proxy_set_header Connection       "upgrade";
        proxy_read_timeout                 300s;
    }

    location /admin/ {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /static/ {
        alias /opt/temple/app/backend/staticfiles/;
        access_log off; expires 30d;
    }

    # ---- Frontend static bundle (Vite build) ----
    location / {
        root /opt/temple/app/frontend/dist;
        index index.html;
        try_files $uri/ /index.html;    # SPA fallback
        add_header Cache-Control "public, max-age=3600";
    }

    # Gzip
    gzip on;
    gzip_types text/plain text/css application/json application/javascript
               application/xml image/svg+xml;
    gzip_min_length 1024;
}

sudo ln -sf /etc/nginx/sites-available/temple /etc/nginx/sites-enabled/temple
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx

```

B.7 HTTPS with Let's Encrypt (free)

```

sudo certbot --nginx -d temple.example.com -d www.temple.example.com \
  --agree-tos -m admin@example.com --redirect --non-interactive

# Auto-renewal is already scheduled by the certbot package.
# Verify:
sudo systemctl status certbot.timer

```

B.8 Database import (from a dump)

```

# From your local machine:
scp -i temple.pem temple_db.sql ubuntu@<ELASTIC_IP>:/home/ubuntu/

# On the EC2 instance:
sed -i 's/utf8mb4_0900_ai_ci/utf8mb4_general_ci/g' ~/temple_db.sql
mysql -u appadmin -p temple < ~/temple_db.sql
rm ~/temple_db.sql    # tidy up

```

B.9 Copy the media/ folder

Uploaded member photos, documents, festival images, etc. live under `backend/temple_proj/media/`. Copy from your old server (or laptop):

```

# On your old server / laptop:

```

```
tar czf media.tgz -C /path/to/old/backend/temple_proj media
scp -i temple.pem media.tgz ubuntu@<ELASTIC_IP>:/home/ubuntu/
```

On the EC2 instance:

```
cd /opt/temple/app/backend/temple_proj
tar xzf ~/media.tgz
sudo chown -R ubuntu:www-data media
sudo chmod -R 755 media
```

C. Environment Files

C.1 backend/.env

Save this at /opt/temple/app/backend/.env (on EC2) or ~/temple/backend/.env (local).
Never commit this file to Git.

```
# ==== Django ====
DJANGO_SETTINGS_MODULE=temple_proj.settings.settings
SECRET_KEY=CHANGE_ME_TO_A_LONG_RANDOM_STRING_MIN_50_CHARS
DEBUG=False
ALLOWED_HOSTS=temple.example.com,www.temple.example.com,127.0.0.1,localhost
CSRF_TRUSTED_ORIGINS=https://temple.example.com,https://www.temple.example.com

# ==== MySQL (MariaDB) ====
DB_NAME=temple
DB_HOST=localhost
DB_PORT=3306
DB_USER=appadmin
DB_PASSWORD=REPLACE_WITH_STRONG_PW

# ==== Media / Static ====
MEDIA_URL=/api/media/
STATIC_URL=/static/

# ==== Business rule ====
PENALTY_PER_MONTH=25

# ==== Optional: S3 for media (if used in production) ====
# AWS_ACCESS_KEY_ID=...
# AWS_SECRET_ACCESS_KEY=...
# AWS_STORAGE_BUCKET_NAME=temple-media
# AWS_S3_REGION_NAME=ap-south-1
# AWS_S3_CUSTOM_DOMAIN=cdn.example.com

# ==== Optional: Email / SMS ====
# EMAIL_HOST=smtp.sendgrid.net
# EMAIL_HOST_USER=apikey
# EMAIL_HOST_PASSWORD=...
# EMAIL_PORT=587
# EMAIL_USE_TLS=True
```

C.2 frontend/.env

Save at /opt/temple/app/frontend/.env (on EC2) or ~/temple/frontend/.env (local).
Vite reads only VITE_* variables.

```
# ==== Backend URL ====
# Production (Nginx serves both frontend + /api on same domain):
VITE_BACKEND_URL=https://temple.example.com

# Local development (backend on port 8000):
# VITE_BACKEND_URL=http://localhost:8000

# ==== UI branding ====
VITE_APP_NAME=Temple Management
VITE_DEFAULT_LOCALE=en-IN
VITE_CURRENCY=INR

# ==== Optional feature flags ====
VITE_ENABLE_WHATSAPP=true
```


VITE_ENABLE_PRINT=true

Note: After changing frontend/.env you must run `yarn build` again (env variables are baked into the JS bundle at build time). After changing backend/.env run `sudo systemctl restart temple-backend`.

D. Post-Install Checklist and Common Issues

D.1 Smoke test

- Site opens over HTTPS at your domain (padlock icon visible).
- Login works with the superuser you created.
- Home page renders with the sidebar; no overlap between sidebar and content.
- GET /api/user/admins_view/ returns 200 (not 500) with the JWT header.
- GET /api/penalty/summary/ returns rate_per_month: 25.0 and a total.
- Uploading a photo on a Member profile shows the file back in the Member List.
- `sudo systemctl status temple-backend` shows 'active (running)'.
- Nginx access log (`/var/log/nginx/access.log`) shows /api/* hitting 127.0.0.1:8000.

D.2 Common issues

502 Bad Gateway on /api/*

Django is not running or crashed. Check `sudo journalctl -u temple-backend -n 100` and `/var/log/temple/backend.err.log`.

Broken images on Member List

The uploaded file is missing on disk. Verify `ls /opt/temple/app/backend/temple_proj/media/`. If empty, re-upload the media folder as in step B.9.

Login returns 204 No Content

Email or password is wrong. Reset from a Django shell:

```
cd /opt/temple/app/backend && source venv/bin/activate
python manage.py shell <<'PY'
from user.models import User
u = User.objects.get(email='admin@gmail.com')
u.set_password('NewStrongPass123')
u.is_superuser = True; u.is_staff = True
u.save()
PY
```

CORS / CSRF errors after HTTPS

Ensure `ALLOWED_HOSTS` and `CSRF_TRUSTED_ORIGINS` in `backend/.env` list the exact https URL, then restart the service.

D.3 Regular maintenance

- **Daily DB backup:** schedule with cron:

```
# /etc/cron.d/temple-backup
0 2 * * * ubuntu mysqldump -u appadmin -p'STRONG_PW' temple | \
  gzip > /var/backups/temple/temple_$(date +%F).sql.gz
```

- **Log rotation:** logrotate is already installed. Add `/etc/logrotate.d/temple` to rotate `/var/log/temple/*.log` weekly.
- **Certificate renewal:** handled automatically by `certbot.timer`.
- **System updates:** `sudo unattended-upgrade -d` once a month.

End of guide. Keep this PDF next to the server for your ops team.